Day 10 — React.js Introduction

Answers and step-by-step practical solution based on the uploaded Day 10 task sheet

Part A: Theory Answers

1. What is React.js?

React.js is a JavaScript library used to build user interfaces, especially for web applications. It lets developers split a page into small reusable parts called components. React updates the parts of the page that need to change when data changes.

2. What are the advantages of React?

Reusable components: Write a component once and use it in multiple places.
Declarative UI: Describe what the interface should look like for a given state or data.
Efficient updates: React compares UI changes and updates the necessary parts of the page.
Large ecosystem: Many libraries and learning resources are available.
Maintainability: Breaking a page into components makes larger interfaces easier to organize.

3. What is a component?

A component is a reusable, independent piece of a user interface. For example, a website can have Header, Footer, Home, ProductCard, and Navbar components. In modern React, components are commonly written as JavaScript functions.

4. What is JSX?

JSX stands for JavaScript XML. It is a syntax that lets you write HTML-like markup inside JavaScript. JSX is transformed into JavaScript that React can use. In JSX, use `className` instead of HTML's `class`, and close tags such as `<img />`.

5. What is a functional component?

A functional component is a JavaScript function that returns React UI, usually written with JSX. Component names should start with a capital letter.

6. What are props?

Props (short for properties) are read-only values passed from a parent component to a child component. They let the same component display different data. Example: `<Header title="My Website" />` passes the title value to Header.

7. What is Vite?

Vite is a frontend development tool that can create and run a React project. It provides a development server and fast updates while coding, and it can build files for production.

8. Explain the src and public folders.

src: Usually contains application source code such as components, CSS, and the main React entry file.
public: Contains static files that should be served as-is, such as a favicon or a file referenced by its public path. Imported images and most app code are commonly kept under src.

9. What is project structure?

Project structure is the organization of folders and files in an application. A clear structure helps developers find components, styles, assets, and entry files quickly.

10. How do components communicate using props?

The parent component supplies values as attributes. The child receives them through its props parameter. For example, `<Home username="Shruti" />` and inside Home use `function Home({ username })`.

Part B: Practical — Create the React App with Vite

1. **1. Open VS Code** and open the terminal using Terminal → New Terminal.
2. **2. Check Node.js and npm:** run `node -v` and `npm -v`. If both show version numbers, continue. Otherwise install the current Node.js LTS release and reopen the terminal.
3. **3. Create the project:** run `npm create vite@latest day10-react -- --template react`. If prompted to install a package, enter `y`.
4. **4. Enter the project folder:** run `cd day10-react`.
5. **5. Install dependencies:** run `npm install`.
6. **6. Start the development server:** run `npm run dev`. Open the local URL printed in the terminal, usually `http://localhost:5173`. Keep the terminal running.
7. **7. In VS Code Explorer**, open the `src` folder. You will edit the files there.

Part C: Create Header, Home, and Footer Components

Inside `src`, create a folder named `components`. Create these three files inside it.

1. src/components/Header.jsx

```
function Header({ title }) {
  return (
    <header className="header">
      <h1>{title}</h1>
      <p>Learning React with Vite</p>
    </header>
  );
}

export default Header;
```

2. src/components/Home.jsx

```
function Home({ username }) {
  return (
    <main className="home">
      <h2>Welcome, {username}!</h2>
      <p>This is my first React application created with Vite.</p>
    </main>
  );
}

export default Home;
```

3. src/components/Footer.jsx

```
function Footer() {
  return (
    <footer className="footer">
      <p>© 2026 My React Website</p>
    </footer>
  );
}

export default Footer;
```

Part D: Connect the Components

Replace the contents of `src/App.jsx` with:

```
import Header from './components/Header';
import Home from './components/Home';
import Footer from './components/Footer';
import './App.css';

function App() {
  return (
```

```
    <div className="app">
      <Header title="My React Website" />
      <Home username="Shruti" />
      <Footer />
    </div>
  );
}

export default App;
```

Replace the contents of `src/App.css` with:

```
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #222;
  background: #f4f6f8;
}

.app {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.header, .footer {
  background: #16324f;
  color: white;
  text-align: center;
  padding: 20px;
}

.home {
  flex: 1;
  text-align: center;
  padding: 60px 20px;
}

.footer { margin-top: auto; }
```

Check src/main.jsx

Vite's default React project normally already has this entry file. It should import App and render it inside the root element. If your file is different, use:

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Part E: Expected Output and Final Checklist

When you open the local Vite URL, the page should show a dark header with “My React Website”, a welcome message for Shruti in the center, and a dark footer at the bottom.

- React application created using Vite.
- src and public folders identified.
- Header, Home, and Footer components created.
- Props used to pass title and username from App to child components.
- Application runs locally with npm run dev.

Important: Component filenames and import paths must match exactly. Save the files and check the browser and terminal if an error appears.